PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

ALGORITMOS AVANZADOS

Primer Examen
(Segundo Semestre 2025)

Duración: 2h 50 min.

- **No puede utilizar apuntes, solo hojas sueltas en blanco.**
- En cada función el alumno deberá incluir, a modo de comentario, la forma de solución que utiliza para resolver el problema. De no incluirse dicho comentario, el alumno perderá el derecho a reclamo en esa pregunta.
- No puede emplear plantillas o funciones no vistas en los cursos de programación de la especialidad.
- Los programas deben ser desarrollados en el lenguaje C++. Si la implementación es diferente a la estrategia indicada o no la incluye, la pregunta no será corregida.
- Un programa que no muestre resultados coherentes y/o útiles será corregido sobre el 50% del puntaje asignado a dicha pregunta.
- Debe utilizar comentarios para explicar la lógica seguida en el programa elaborado. El orden será parte de la evaluación.
- Se utilizarán herramientas para la detección de plagios, por tal motivo si se encuentran soluciones similares, se anulará la evaluación a todos los implicados y se procederá con las medidas disciplinarias dispuestas por la FCI.
- **Solo está permitido acceder a la plataforma de PAIDEIA, cualquier tipo de navegación, búsqueda o uso de herramientas de comunicación se considera plagio por tal motivo se anulará la evaluación y se procederá con las medidas disciplinarias dispuestas por la FCI.**
- Para esta evaluación solo se permite el uso de las librerías **iostream, iomanip, climits cmath, fstream, vector, algorithm, string o cstring**
- Su trabajo deberá ser subido a PAIDEIA.
- **Es obligatorio usar como compilador CLion.**
- Los archivos deben llevar como nombre su código de la siguiente forma codigo_EX1_P# (donde # representa el número de la pregunta a resolver)

Pregunta 1 (10 puntos)

Problema: Sum Sudoku (KenKen simplificado)

Se tiene una cuadrícula de tamaño n × n que debe completarse con números del 1 al n cumpliendo las siguientes condiciones:

a) En cada fila los números no deben repetirse.
b) En cada columna los números no deben repetirse.
c) La cuadrícula está dividida en *k* **regiones** (o bloques). Cada región R_i tiene una **suma objetivo** S_i. La suma de los valores de las celdas que pertenecen a la región R_i debe ser exactamente S_i. No se permite repetir números dentro de una misma región, aunque estén en diferentes filas o columnas. Además, cada celda del tablero solo puede pertenecer a una región.

Formato de Entrada:

La primera línea contiene dos enteros: *n* y *k*, donde n es el tamaño de la cuadrícula y *k* el número de regiones. Las siguientes *k* líneas describen cada región. Cada línea tiene el formato:

$$S\ m\ r_1\ c_1\ r_2\ c_2\ \ldots\ r_m\ c_m$$

donde:

S es la suma objetivo de la región.

m es el número de celdas de la región.

(r_j, c_j) son las coordenadas de cada celda (con índices iniciando en cero).

Formato de Salida:

- Si existe una solución, se imprimen *n* líneas con *n* enteros separados por espacios.
- Si no existe solución, se imprime exactamente la línea:
  1. NO HAY SOLUCION

Se le pide a Usted que elabore una propuesta de solución al problema del Sum Sudoku (KenKen simplificado) usando una de las técnicas vista en el curso empleando para ello el lenguaje C++.

1. Casos de Prueba

Ejemplo 1

Entrada

```
3 4
4 2 0 0 0 1
3 1 0 2
5 2 1 0 2 0
7 4 1 1 1 2 2 1 2 2
```

Salida esperada

```
1 3 2
2 1 3
3 2 1
```

Ejemplo 2

Entrada

```
4 6
3 2 0 0 0 1
9 3 0 2 0 3 1 3
5 2 1 0 2 0
8 3 2 1 3 0 3 1
10 4 2 2 2 3 3 2 3 3
```

Salida esperada

```
1 2 3 4
3 4 1 2
2 1 4 3
4 3 2 1
```

Ejemplo 3

Entrada

```
5 9
3 2 0 0 0 1
12 3 0 2 0 3 0 4
5 2 1 0 1 1
9 2 1 2 1 3
3 2 1 4 2 4
12 3 2 0 2 1 3 1
8 3 2 2 3 2 3 3
8 3 2 3 3 4 4 4
11 4 4 0 4 1 4 2 4 3
```

Salida esperada

```
1 2 3 4 5
2 3 4 5 1
3 4 5 1 2
4 5 1 2 3
5 1 2 3 4
```

Pregunta 2 (10 puntos)

Una empresa dedicada a la instalación de cerámicos (TUPIATENCHAPA) adquiere robots para realizar este trabajo, debido a que la tarea de colocar o enchapar cerámicos es bastante pesada y los robots son difíciles de mover, se debe asignar los trabajos de forma consecutiva a cada unidad, por lo cual el orden de las tareas que se brindan como dato no se deben modificar y un robot no puede terminar un trabajo y luego continuar con otro que no se encuentre consecutivamente. La empresa busca minimizar el tiempo que demoran los robots en realizar el enchape, por eso se ha elegido como indicador representativo de la asignación de trabajos a la suma de tareas que le demanda mayor tiempo a un robot. A continuación, un ejemplo:

Caso 1:

Datos de entrada:

Si cuenta con los siguientes trabajos dados en minutos: {5, 10, 30, 20, 15} y se tiene *k*=2 robots

Respuesta:

El tiempo mínimo representativo en que los 2 robots empacharan el piso será 45 min. El primer robot realizará los trabajos 0, 1, 2 que da 45 min y el segundo robot realizará los trabajos 3 y 4

Caso 2:

Datos de entrada:

Si cuenta con los siguientes trabajos dados en minutos: {5, 10, 30, 20, 15} y se tiene *k*=3 robots

Respuesta:

El tiempo mínimo representativo en que los 3 robots empacharan el piso será 35 min. El primer robot realizará los trabajos 0, 1 que da 15 min, el segundo robot realizará el trabajo 2 y el tercer robot realizará los trabajos 3 y 4 que suma 35 min.

Se le solicita que desarrolle un programa en C++ que empleando programación dinámica obtenga el tiempo representativo de cada asignación.

Para que la solución sea válida se debe mostrar la matriz de soluciones parciales, recuerde que solo debe emplear iteraciones, si la respuesta emplea recursión o no muestra la matriz de soluciones parciales la pregunta queda anulada.

Pregunta 3 (10 puntos)

En una embotelladora de bebidas gaseosas, se tienen M líneas de producción, cada una de las cuales TechBoard S.A. fabrica placas electrónicas personalizadas para distintos clientes del sector automotriz y de telecomunicaciones. La planta posee M = 4 máquinas de ensamblaje automatizadas, cada una con características distintas (velocidad, precisión y tiempo de calibración) y que pueden ejecutar cualquier tarea, pero con tiempos de procesamiento diferentes según el tipo de tarea asignada. El objetivo del jefe de producción es programar n = 10 órdenes de ensamblaje (tareas independientes) en las 4 máquinas, de forma que el tiempo total de finalización (makespan) sea el menor posible, es decir, que todas las órdenes se terminen lo antes posible.

En la siguiente tabla se ve el detalle de las máquinas:

Máquina	Tipo	Capacidad relativa	Observaciones
M_1	Ensambladora de precisión	Lenta pero muy estable	Ideal para microcomponentes
M_2	Ensambladora multifunción	Velocidad media	Requiere calibración corta entre tareas distintas
M_3	Ensambladora rápida	Alta velocidad	Menor precisión; no apta para piezas delicadas
M_4	Ensambladora robotizada	Rápida y flexible	Alta eficiencia, pero con sobrecosto energético

Las ordenes de producción son las siguientes con los tiempos estimados en minutos dependiendo en qué ensambladora son procesadas:

Tarea	Descripción	M_1	M_2	M_3	M_4
T_1	Placa para sensores automotrices	6	4	3	2
T_2	Controlador IoT industrial	7	6	4	3
T_3	Placa base de router Wi-Fi	4	3	2	2
T_4	Módulo GPS de precisión	8	5	11	4
T_5	Circuito de audio	5	3	2	3
T_6	Panel de control automotriz	9	6	5	4
T_7	Tarjeta de sensores ambientales	7	4	3	3
T_8	Placa de cámara térmica	6	5	4	3
T_9	Controlador de motor eléctrico	8	5	4	4
T_{10}	Módulo de comunicación 5G	9	6	5	3

Se tienen como restricciones operativas:

- Cada tarea debe asignarse a exactamente una máquina.
- Cada máquina puede ejecutar una tarea a la vez.
- Las tareas son independientes entre sí (no hay precedencias).
- El tiempo total de una máquina es la suma de los tiempos de las tareas asignadas a ella.

El gerente de planta debe decidir cómo distribuir las 10 órdenes de producción entre las 4 ensambladoras. Si distribuye mal las tareas, una máquina podría estar saturada mientras otras quedan ociosas, aumentando el tiempo total de entrega. Buscar manualmente la asignación óptima es impracticable: el número de combinaciones es superior al millón.

Por eso, TechBoard evalúa el uso de algoritmos voraces para resolver el problema. Desarrolle un algoritmo voraz en C++ que lo resuelva.

Al finalizar el examen, comprima la carpeta de su proyecto empleando el programa Zip que viene por defecto en el Windows, **no se aceptarán los trabajos compactados con otros programas como RAR, WinRAR, 7zip o similares**. Luego súbalo a la tarea programa en Paideia para este examen.

Profesores del curso:

Manuel Tupia
Rony Cueva
Andrés Melgar

San Miguel, 14 de octubre del 2025